

Git & GitHub

The Complete Guide — Beginner to Proficient

Every core command explained — what it does, how to use it, and when — with a running scenario, diagrams, and simple → moderate → advanced coverage.

A practical, hands-on reference

How to Read This Guide

This guide takes you from never having used Git to being genuinely proficient. You do not need to read it all at once — work through it in order the first time, then use the cheat sheets at the end as a day-to-day reference.

Every important command is explained in three parts: **what it does** (the idea), **how to use it** (the exact command), and **when to use it** (the situation that calls for it). Commands are grouped by level so you can grow steadily:

Simple — the everyday commands you will use within your first hour (install, config, init, add, commit, push, pull).

Moderate — the team workflow: branches, merges, remotes, pull requests, and undoing mistakes safely.

Advanced — the power tools that make you fast and confident: rebase, cherry-pick, stash, tags, reflog, bisect, and more.

A running **Scenario** follows a fresher named Ravi building a website called “my-first-project”, so each command is shown solving a real problem. Yellow **Note** boxes flag the mistakes beginners actually make.

Convention used throughout: text in this style is something you type. Replace anything in `<angle-brackets>` with your own value — e.g. `<branch-name>` means type your actual branch name, without the brackets.

1. What is Git & GitHub?

Imagine writing an important assignment and saving copies called `final.doc`, `final2.doc`, and `final_USE_THIS_ONE.doc`. You lose track of which is newest, you cannot see what changed between them, and if two friends edit at once you have no safe way to combine their work. Version control solves exactly this problem for code — automatically, reliably, and for any number of people.

Git — the tool on your computer

Git is a **distributed version control system**: a program that records snapshots of your project over time. Each snapshot (a **commit**) remembers exactly what every file looked like and who changed what. You can review history, return to any earlier version, and merge changes from many people without overwriting each other. “Distributed” means every copy carries the full history — you can work and commit with no internet at all.

GitHub — the shared home in the cloud

GitHub is a **website that hosts Git repositories**. Git works perfectly on your own machine, but GitHub adds a shared place so a team can push and pull changes, review each other’s work through pull requests, track issues, and collaborate from anywhere. GitLab and Bitbucket do the same job — the Git commands in this guide work with all of them.

Remember it like this: Git is the engine on your laptop; GitHub is the shared garage on the internet where everyone parks their copy. You can use Git with no GitHub — but using GitHub always uses Git underneath.

Why learn it?

- It is the industry standard — expected knowledge in almost every software job and interview.
- It makes mistakes safe — you can always recover an earlier working version.
- It lets many people build one project together without chaos.
- Your public GitHub profile is a portfolio employers can see.

Scenario: We follow Ravi, a fresher, building a small website “my-first-project”. We begin with an empty folder on his laptop and finish with his code on GitHub, a feature built on a branch, a mistake safely undone, and a teammate contributing through a pull request.

2. Installing & Configuring Git

Git is free and runs on every major operating system. Install it once, and every project on your machine can use it.

Windows

1. Download “Git for Windows” from git-scm.com.
2. Run the installer — the default options are fine for a beginner.

It also installs **Git Bash**, a terminal where you type Git commands. Use it for everything here.

macOS

The simplest route is the **Homebrew** package manager:

```
brew install git
```

No Homebrew? Just typing `git --version` in Terminal prompts macOS to install the developer tools, which include Git.

Linux (Debian / Ubuntu)

```
sudo apt-get update
sudo apt-get install git
```

On Fedora or Red Hat use "sudo dnf install git". Many Linux systems ship with Git already installed.

Verify the install

```
git --version

# Example output:
git version 2.56.0
```

A version number means Git is ready. “Command not found” means the install did not finish — reinstall and open a fresh terminal.

git config — tell Git who you are

What: records your identity so every commit is labelled with your name and email. **When:** once per machine, before your first commit.

```
git config --global user.name "Ravi Kumar"
git config --global user.email "ravi@example.com"

# make "main" the default branch for new repositories
git config --global init.defaultBranch main

# optional quality-of-life settings
git config --global color.ui auto          # colourful output
git config --global core.editor "code --wait" # use VS Code for messages
```

The `--global` flag means “for every project on this computer”. Review your settings any time with `git config --list`. Use the same email you will use on GitHub so commits link to your account.

Connecting to GitHub (authentication)

To push code to GitHub you need a free account and a way to prove it is you. Two common methods:

HTTPS + Personal Access Token (easiest to start) — clone/push over https; when asked for a password, paste a token you generate in GitHub under Settings → Developer settings → Personal access tokens. (GitHub no longer accepts your account password here.)

SSH keys (smooth once set up) — generate a key pair once, add the public key to GitHub, and pushing never asks for a password again.

```
# Generate an SSH key (press Enter to accept defaults)
ssh-keygen -t ed25519 -C "ravi@example.com"

# Copy the PUBLIC key into GitHub -> Settings -> SSH and GPG keys
cat ~/.ssh/id_ed25519.pub

# Test the connection
ssh -T git@github.com
```

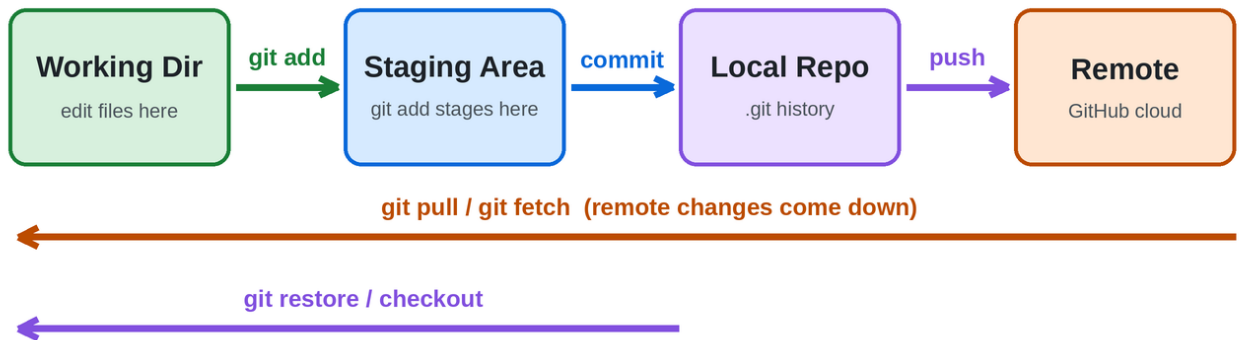
Your Personal Access Token and SSH private key are secrets — treat them like passwords. Never commit them, paste them into code, or share them.

Scenario: Ravi installs Git for Windows, opens Git Bash, and confirms git version 2.56.0. He sets his name and email with git config, creates a free GitHub account with the same email, and adds an SSH key. His laptop is ready to talk to GitHub.

3. How Git Thinks: The Three Areas

Almost every Git command makes sense once you picture where your work lives. There are three local areas plus the remote on GitHub, and commands move your work between them.

The Git Mental Model: Three Areas + Remote



git status shows what changed in the Working Directory and Staging Area at any time.

Figure 1 — Changes flow left to right: edit files, git add to stage, git commit to save history, git push to GitHub. Commands below the boxes bring work back the other way.

Area	What it is
Working Directory	Your real files on disk, where you edit. Changes here are “untracked” or “modified” until you stage them.
Staging Area (Index)	A holding tray for the exact changes that will go into your next commit. You fill it with git add.
Local Repository	The permanent history stored in the hidden .git folder. git commit saves the staging area here as a snapshot.
Remote (GitHub)	The shared copy in the cloud. git push sends commits up; git pull brings others’ commits down.

A commit is identified by a unique **hash** (like a3f9c21). **HEAD** is a pointer to “where you are now” — usually the latest commit on your current branch. You will see both terms throughout this guide.

4. The Everyday Workflow

These are the commands that make up the daily heartbeat of Git. We run them in order on Ravi’s project so you see how they connect.

git init

What: turns a plain folder into a Git repository by creating a hidden .git folder. **How:** run it once inside your project. **When:** starting a brand-new project that is not yet tracked.

```
mkdir my-first-project
```

```
cd my-first-project
git init
```

git status

What: shows what has changed, what is staged, and what is untracked. **How:** run it any time. **When:** constantly — it is your “where am I?” command.

```
echo "<h1>Hello</h1>" > index.html
git status

# Git reports index.html as "Untracked"
```

git add

What: moves changes into the staging area, choosing what goes into the next commit. **How:** name files, or use a dot for everything. **When:** right before committing.

```
git add index.html      # stage one file
git add .               # stage everything changed
git add -p              # stage interactively, chunk by chunk
```

git add -p (patch mode) lets you stage parts of a file and leave the rest — perfect for splitting unrelated edits into separate, clean commits.

git commit

What: saves everything staged as a permanent snapshot with a message. **How:** use -m for the message. **When:** after each logical, self-contained change.

```
git commit -m "Add homepage with a hello heading"

# stage all tracked changes AND commit in one step
git commit -am "Fix typo in heading"
```

Write messages in the present tense describing what the change does: “Add login form”, not “added stuff”. Keep each commit focused on one idea — future-you will thank present-you.

git log

What: shows the commit history, newest first. **How:** plain, or with flags for a compact view. **When:** to review what happened or find a commit hash.

```
git log                # full history
git log --oneline      # one compact line per commit
git log --oneline --graph --all  # visual branch graph
```

git diff & git show

What: git diff shows line-by-line changes you have NOT staged yet; git show displays a single commit in full. **When:** to review exactly what you changed before staging, or inspect an old commit.

```
git diff              # unstaged changes
git diff --staged     # what is staged, ready to commit
git show <commit-hash> # full details of one commit
```

.gitignore — keep junk out

What: a plain text file listing patterns Git should ignore (build output, secrets, dependencies). **When:** add it at the very start of a project. **Why:** it stops you accidentally committing secrets or huge generated folders.

```
# .gitignore (one pattern per line)
node_modules/      # installed dependencies
.env               # secret keys and passwords
*.log              # log files
dist/              # build output
```

Scenario: Ravi runs `git init`, creates `index.html`, and `git status` shows it untracked. He adds a `.gitignore` for `.env`, runs `git add .`, then `git commit -m "Add homepage"`. `git log --oneline` now shows his first commit. He has a tracked project with real history.

5. Working with GitHub

So far everything is on Ravi's laptop. Now we connect to GitHub so the project is backed up and shareable. The link between your local repo and the GitHub copy is called a remote, and it is conventionally named origin.

git remote

What: links your local repo to a repo on GitHub. **How:** first create an empty repo on GitHub, then add its URL. **When:** once, to connect a local project to the cloud.

```
# HTTPS
git remote add origin https://github.com/ravi/my-first-project.git
# or SSH
git remote add origin git@github.com:ravi/my-first-project.git

git remote -v          # list remotes and verify
```

git push

What: uploads your local commits to GitHub. **How:** the first push uses `-u` to link local and remote branches. **When:** after committing, to share or back up your work.

```
git push -u origin main  # first time
git push                 # every time after
```

git clone

What: downloads an existing GitHub repo — all files and full history — and sets up origin automatically. **When:** to start working on a project that already lives on GitHub.

```
git clone https://github.com/someone/cool-project.git
```

git pull vs git fetch

What: both download commits others pushed. `git fetch` downloads but does NOT change your files (you look first); `git pull` downloads AND merges into your current branch. **When:** pull at the start of each day; fetch when you want to inspect before merging.

```
git fetch          # download only, merge later
git pull           # download AND merge now
```

Build the habit: `git pull` before you start working each day, so you begin from the latest version and avoid conflicts later.

Scenario: Ravi creates “my-first-project” on GitHub, runs `git remote add origin ...`, then `git push -u origin main`. He refreshes the page and sees `index.html` online. His code is safely in the cloud.

6. Branching & Merging

A branch is a separate line of work. It lets you build a feature without disturbing the known-good version, then combine it back when ready. This is the heart of real-world Git.

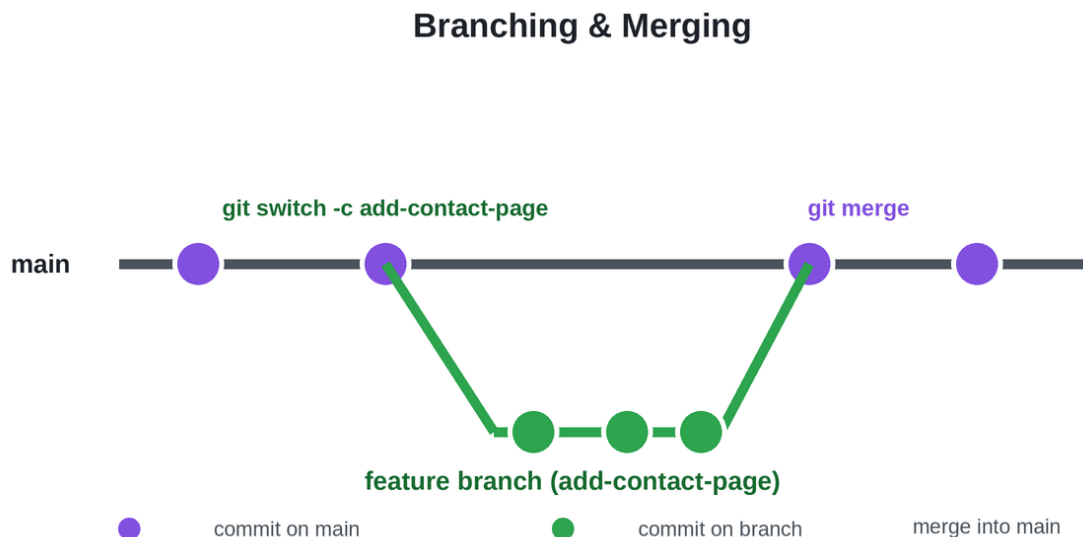


Figure 2 — Branch off main to build a feature, make commits on the branch, then merge it back into main when it is ready.

Why branches exist

Think of `main` as the always-working version. Editing it directly while a feature is half-finished breaks the live version. Instead you branch off, experiment freely, and merge back only once it works. Many people can each work on their own branch at the same time.

git branch & git switch

What: `git branch` creates/lists branches; `git switch` moves between them (older guides use `git checkout`). **When:** start every new feature or fix on its own branch.

```
git branch                # list branches (* marks current)
git switch -c add-contact-page # create AND switch in one step
git switch main           # move back to main

# the older equivalent you will still see:
git checkout -b add-contact-page
```

git merge

What: brings the commits from one branch into another. **How:** switch to the receiving branch (usually `main`) and merge the feature in. **When:** the feature is complete and committed.

```
git switch main
git merge add-contact-page
git branch -d add-contact-page # delete the finished branch
```

Handling merge conflicts

A conflict happens when two branches changed the SAME lines differently — Git cannot guess which to keep, so it asks you. It marks the spot in the file:

```
<<<<<< HEAD
the version currently on your branch
=====
the version coming from the other branch
>>>>>> add-contact-page
```

To resolve: open the file, delete the <<<<, =====, >>>> marker lines, keep the final text you want (which may blend both), then stage and commit.

```
git add index.html
git commit -m "Resolve merge conflict in homepage"
```

Conflicts feel scary the first time but are completely normal. Nothing is lost — Git simply hands you the decision. Pulling often keeps conflicts small and rare.

Scenario: Ravi makes branch add-contact-page, builds contact.html, commits, switches to main, and merges cleanly. Later he and a teammate both edit the homepage title; the merge conflicts. Ravi opens the file, keeps the agreed wording, removes the markers, and commits. Resolved.

7. Undoing Mistakes Safely

Git's superpower is that almost anything can be undone. The skill is picking the right command for the situation — some are safe on shared work, some are not.

git restore

What: discards uncommitted edits or unstages files. **When:** you changed a file and want it back as it was at the last commit.

```
git restore index.html          # discard edits to this file
git restore --staged index.html # unstage, but keep the edits
```

git restore <file> permanently discards uncommitted edits to that file — it cannot be undone. Be sure before you run it.

git revert — the safe undo

What: creates a NEW commit that cancels an earlier one. Because it adds history rather than deleting it, it is safe for commits you already pushed. **When:** undoing something already on GitHub / shared with others.

```
git revert <commit-hash>
```

git reset — rewind the branch

What: moves your branch pointer back to an earlier commit. **How:** --soft keeps your changes staged, --mixed (default) keeps them unstaged, --hard discards them. **When:** cleaning up local commits you have NOT pushed yet.

```
git reset --soft HEAD~1    # undo last commit, KEEP changes staged
git reset HEAD~1           # undo last commit, keep changes unstaged
git reset --hard HEAD~1    # undo last commit AND discard its changes
```

Never use `git reset` (especially `--hard`) on commits already pushed to a shared branch — it rewrites history others may rely on. For anything already on GitHub, use `git revert` instead.

`git clean` — remove untracked files

What: deletes untracked files/folders Git is not managing. **When:** clearing out build junk. Always dry-run first.

```
git clean -n    # DRY RUN: show what would be deleted
git clean -fd   # actually delete untracked files and folders
```

`git reflog` — the ultimate safety net

What: a log of everywhere HEAD has been — even commits you “lost” via a bad reset. **When:** you think you destroyed work; `reflog` can usually bring it back.

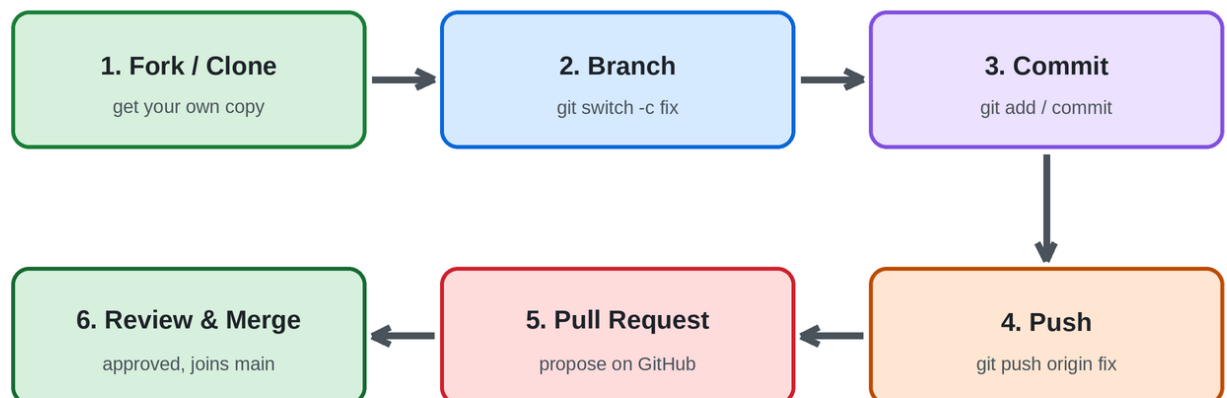
```
git reflog    # list recent HEAD positions
git reset --hard <hash> # jump back to a recovered state
```

Scenario: Ravi deletes text in `index.html` before committing — `git restore index.html` brings it back. Later he pushes a bad commit; instead of rewriting shared history he runs `git revert` to add a clean undo commit. When a careless `git reset --hard` loses a commit, `git reflog` finds its hash and he restores it.

8. Collaborating on GitHub

GitHub is where Git becomes a team sport. Two ideas unlock almost all collaboration: forks and pull requests.

The GitHub Collaboration Flow



A pull request shows exactly what changed and lets teammates review it line by line.

Figure 3 — The contribution flow: get a copy, branch, commit, push, open a pull request, then review and merge.

Forking

A **fork** is your own copy of someone else's GitHub repository under your account. You use it to contribute to a project you cannot write to directly (common in open source): fork it, clone your fork, change it, then propose the change back.

Pull requests (PRs)

A **pull request** is a proposal: "here are my changes — please review and merge". It shows exactly what changed, lets teammates comment line by line, runs any automated checks, and records the discussion. PRs are how code is reviewed before joining the main project.

The typical contribution flow

3. Fork the repo (open source) or just clone it (your own team's repo).
4. Create a branch: `git switch -c fix-readme-typo`.
5. Make edits, then `git add` and `git commit`.
6. Push the branch: `git push -u origin fix-readme-typo`.
7. On GitHub, click "Compare & pull request", describe it, and open the PR.
8. A reviewer comments or approves; once approved, the PR is merged into main.
9. Update your local main: `git switch main && git pull`, then delete the finished branch.

Good collaboration habits

- Keep PRs small and focused — they are far easier to review and merge.
- Write a clear PR description: what changed and why.
- Pull main and rebase/merge before opening a PR to reduce conflicts.
- Use Issues to track bugs and tasks; link them from PRs (e.g. "Fixes #42").

Scenario: Ravi spots a typo in an open-source project. He forks it, clones his fork, branches `fix-readme-typo`, commits the fix, pushes, and opens a pull request. A maintainer approves and merges it — his first open-source contribution.

PART C — ADVANCED: BECOME POWERFUL & FAST

These commands are not needed on day one, but they are what separate a confident Git user from a beginner. Learn them once you are comfortable with everything above.

9. Reshaping History

git stash

What: tucks uncommitted changes aside so your working directory is clean, then restores them later. **When:** you are mid-change but must urgently switch branches to fix something else.

```
git stash          # set current changes aside
git stash list     # see saved stashes
git stash pop      # restore the latest and remove it from the list
git stash apply    # restore but keep it in the list
git stash -u       # also stash untracked files
```

git commit --amend

What: rewrites the MOST RECENT commit — to fix its message or add a forgotten file. **When:** you just committed and immediately noticed a small mistake. **Caution:** only on commits you have NOT pushed.

```
git add forgotten-file.css
git commit --amend -m "Add homepage with styles"
```

git rebase — a cleaner history

What: replays your branch's commits on top of another branch, producing a straight, linear history instead of a merge bubble. **When:** updating a feature branch with the latest main before a PR.

Caution: it rewrites commit hashes — never rebase commits others already pulled.

```
git switch add-contact-page
git rebase main          # replay my commits on top of latest main

# if a conflict stops it: fix files, then
git add <file>
git rebase --continue
git rebase --abort       # or bail out and go back
```

Golden rule of rebase: rebase local, private commits freely; never rebase (or reset) commits that are already shared on a branch others use. For shared history, merge or revert instead.

Interactive rebase — tidy up before a PR

What: lets you squash several messy commits into one, reword messages, reorder, or drop commits.

When: polishing a branch so reviewers see clean, logical commits.

```
git rebase -i HEAD~3      # edit the last 3 commits

# In the editor, change "pick" to:
#   squash (s) = fold into the previous commit
#   reword (r) = change the message
#   drop (d) = remove the commit entirely
```

git cherry-pick

What: copies ONE specific commit from another branch onto your current branch. **When:** you need just one fix from elsewhere without merging the whole branch.

```
git cherry-pick <commit-hash>
```

git tag — mark releases

What: puts a permanent, friendly label on a commit (like v1.0.0). **When:** marking released versions so you can find them instantly later.

```
git tag -a v1.0.0 -m "First public release"
git push origin v1.0.0    # tags are not pushed by default
git tag                  # list tags
```

Scenario: Before opening his PR, Ravi has three scrappy commits ("wip", "fix", "typo"). He runs `git rebase -i HEAD~3`, squashes them into one clean "Add contact page" commit, rebases onto the latest main, and pushes. His reviewer sees one tidy commit. After release he tags it v1.0.0.

10. Inspecting & Investigating

git blame

What: shows who last changed each line of a file, and in which commit. **When:** understanding why a line exists or who to ask about it (not for assigning blame!).

```
git blame index.html
```

git bisect — find the commit that broke it

What: a binary search through history to pinpoint the exact commit that introduced a bug. **When:** "it worked last week, broken now, no idea which change did it".

```
git bisect start
git bisect bad           # the current commit is broken
git bisect good v1.0.0   # this old commit worked
# Git checks out a middle commit; you test and mark it:
git bisect good          # or: git bisect bad
# ...repeat until Git names the first bad commit...
git bisect reset         # finish and return to where you were
```

Comparing across history

```
git diff main..add-contact-page # differences between two branches
git log main..add-contact-page  # commits on the branch not in main
git show HEAD~2                # look at a commit 2 steps back
```

A few more worth knowing

`git remote prune origin` — clean up references to remote branches that were deleted on GitHub.

`git fetch --all --prune` — refresh everything and tidy stale branches in one go.

`git submodule` — embed one repository inside another (used for shared libraries); learn it when a project you join uses it.

`git worktree` — check out multiple branches into separate folders at once, so you can work on two things without stashing.

Scenario: A bug appears in HelloApp with no obvious cause. Ravi runs `git bisect`, marking versions good or bad as Git walks the history. In six steps Git names the exact commit that introduced the bug. He reads it with `git show`, finds the bad line, and fixes it — minutes instead of hours.

11. Command Cheat Sheet

A fast reference grouped by level. Keep this page open while you work.

Simple — everyday

Command	What it does	When to use it
<code>git init</code>	Start tracking a project in this folder.	New local project.
<code>git clone <url></code>	Download a repo with full history.	Start from an existing repo.
<code>git status</code>	Show changed, staged, untracked files.	Constantly — “where am I?”.
<code>git add <file> / .</code>	Stage changes for the next commit.	Before committing.
<code>git commit -m "..."</code>	Save a snapshot with a message.	After a logical change.
<code>git log --oneline</code>	Compact commit history.	Review history, find a hash.
<code>git diff</code>	Show unstaged line changes.	Check edits before staging.
<code>git push</code>	Upload commits to GitHub.	Share / back up work.
<code>git pull</code>	Download AND merge others’ commits.	Start of each day.

Moderate — team workflow

Command	What it does	When to use it
<code>git switch -c <name></code>	Create a branch and move onto it.	Start a feature or fix.
<code>git switch <name></code>	Move to an existing branch.	Change what you work on.
<code>git merge <branch></code>	Combine a branch into the current one.	Feature is done.
<code>git remote -v</code>	List linked remotes.	Check GitHub connection.
<code>git fetch</code>	Download commits without merging.	Inspect before merging.
<code>git restore <file></code>	Discard uncommitted edits.	Throw away local changes.

Command	What it does	When to use it
<code>git revert <hash></code>	Undo a commit with a new commit.	Undo pushed/shared work.
<code>git reset --hard <ref></code>	Rewind branch, discard changes.	Local-only cleanup.
<code>git reflog</code>	History of where HEAD has been.	Recover “lost” commits.

Advanced — power tools

Command	What it does	When to use it
<code>git stash / pop</code>	Park changes aside, then restore.	Switch tasks mid-change.
<code>git commit --amend</code>	Rewrite the latest commit.	Fix a just-made commit (unpushed).
<code>git rebase <branch></code>	Replay commits for linear history.	Update a feature branch (unshared).
<code>git rebase -i <ref></code>	Squash / reword / reorder commits.	Tidy a branch before a PR.
<code>git cherry-pick <hash></code>	Copy one commit to this branch.	Grab a single fix.
<code>git tag -a <name></code>	Label a commit (e.g. a release).	Mark released versions.
<code>git blame <file></code>	Show who changed each line.	Understand a line’s origin.
<code>git bisect</code>	Binary-search for a bad commit.	Find what introduced a bug.

Stuck on any command? `git help <command>` opens its full manual, and `git <command> -h` shows a quick summary. `git status` is always safe to run and tells you what to do next.

12. Mistakes, Habits & Next Steps

Common mistakes

- Committing secrets — never commit passwords, tokens, or `.env` files. Add them to `.gitignore` from the start.
- Forgetting to pull first — starting on an old version causes avoidable conflicts.

- Giant commits mixing many changes — hard to review or undo. Commit small, logical chunks.
- Vague messages like “fix” or “stuff” — say what actually changed.
- Rebasing or reset --hard on shared commits — rewrites history others have. Use revert for shared work.
- Working directly on main — branch for every feature so main stays stable.

Good habits

- Run git status often — it tells you exactly where you are.
- Commit early and often, with clear present-tense messages.
- Add a .gitignore and a README at the start of every project.
- Pull before you push; branch for every feature or fix; keep PRs small.
- Tag your releases so you can always return to a known-good version.

Where to go next

- Practise on a throwaway repo — try branching, merging, rebasing, and deliberately causing a conflict to resolve it.
- Learn your team’s branching model (e.g. GitHub Flow: branch → PR → review → merge).
- Explore GitHub Actions to run tests automatically on every push (continuous integration).
- Read the official docs at git-scm.com and GitHub’s guides when you want depth on any command.

Scenario: A month in, Ravi’s routine is automatic: pull, branch, make small clear commits, rebase to tidy up, push, open a PR. His main branch is always working, secrets never leak thanks to .gitignore, he recovers mistakes with reflog, and he tags each release. He is genuinely proficient with Git and GitHub.

You now know Git from first install to advanced history surgery. Keep this guide handy, practise often, and the commands will become second nature.